Software Requirements Specification

Version 1.0.0

4/10/2026

A Role-Based Collaborative System for Software Release and Bug Tracking

<<Any comments inside double brackets, such as these, are *not* part of this SRS but are comments upon this SRS example to help the reader understand the point being made>>

Table of Contents

1.0 Introduction

is a collaborative software development platform designed to streamline the lifecycle of code updates from submission to deployment. It provides a structured pipeline that enables teams to manage code contributions, testing, reviewing, and release processes in an organized and automated way.

The platform allows project owners to create projects and invite team members via email, assigning specific roles such as developers, testers, and reviewers. Each role plays a key part in maintaining code quality and ensuring reliable releases.

At the core of ReleaseForge is a controlled pipeline workflow:

Push Code → Test → Review → Merge

Developers can submit code updates along with associated tasks and bug fixes. Once submitted, the system automatically triggers the testing phase, where testers define and execute test cases, marking results as pass or fail. Any modification to the code requires tests to be re-executed, ensuring consistency and reliability.

Only updates that successfully pass all tests can proceed to the review stage, unless overridden by a team leader. Reviewers evaluate the update and can approve, reject, request changes, or roll it back to previous stages with feedback. Approved updates are merged into the project and included in a new release.

The platform also maintains strong traceability between code updates, tasks, and bugs. Completed updates automatically resolve linked tasks and bug reports, while reverting updates restores their open status.

Overall, it enhances team collaboration, enforces quality control, and provides a clear, trackable workflow for managing software releases efficiently.

2.0 Functional Requirements

Prioritization Method: **MoSCoW Method**

2.1 Account Creation, Verification

Priority: Must have

Release: 1.0.0

User Requirement Specification

The user should be able to sign up for the platform using email and password, and enter their first and last name.

System Requirement Specification

2.1.1 The platform should provide an interface to the user for signing up, using email & password, and entering their first and last name.

2.1.2 The platform should validate input fields:

- Email is valid and not used before
- Password, the re-entered password with the entered password
- First and last names are not empty

2.1.3 The platform shall redirect the user to a welcome page after successful account creation.

2.1.4 If the user signed up, the platform must verify their accounts via email by sending a verification link and asking the user to enter the link

2.1.4.1 If anything went wrong during verification, the user can ask for the link to be resent.

2.1.4.2 Not completing verification prevents the user from joining a project or creating one.

2.1.5 After successful sign-up, the user is redirected to the system with restricted access until account verification is complete.

2.2 Project Creation

Priority: Must have

Release: 1.0.0

User Requirement Specification

Any user with an account should be able to create a new project to manage the project workflow.

System Requirement Specification

2.2.1 The system must provide a project creation interface for the Project Owner.

2.2.2 The system must provide an interface to ask the user to fill in project details:

- Project name (required).
- Project Logo (required, 100x100 pixel, any image format
- Brief description (optional, plain text).
- Current Version (optional, 2 floating-point number format).

2.2.3 The system must validate project inputs before submission:

- Project name must not be empty
- If the current version is empty, the default is 0.0.0

2.2.4 Upon successful creation, the system must automatically assign the creator user as the Project Owner with full access permissions.

2.2.5 Upon successful creation, the system shall redirect the Project Owner to the newly created project dashboard.

2.3 Sending invitations to join the project

Priority: Must have

Release: 1.0.0

User Requirement Specification

Project owners should be able to invite users to join a project via email and assign roles to them. Invited users should be able to accept or decline invitations.

System Requirement Specification

2.3.1 The project owner must be able to add new members to the project team by inviting them to the project.

2.3.2 The system must require the following information when sending an invitation::

Invited user Email

Invite Message [Optional, maximum 250 characters, accepts plain text]

Role (Developer, Tester, Reviewer, Team Leader)

<<Permission and description of each role detailed in table x>>

2.3.3 The system must validate that the entered email address is in a valid format.

2.3.4 The system must prevent sending duplicate invitations to the same email for the same project.

2.3.5 The system must generate a unique invitation link for each invitation.

2.3.6 The system must mark the invitation status as:

- Pending
- Accepted
- Expired
- Canceled

2.3.7 The system must automatically revoke invitation links after 13 days.

2.3.8 The system should allow the project owner to manage the invitations.

<<Detailed in Requirement: 2.4>>

2.4 Managing invitations to the project

Priority: Must have

Release: 1.0.0

User Requirement Specification

The project owner can view and manage all sent invitations by canceling or resending them.

System Requirement Specification

2.4.1 The project owner must be able to manage all sent invitations from the invitations list in the project. Detailed in **Table 2.4.1**.

Manage sent invitations		
Action	**Precondition**	**Postcondition**
See all sent invitations	The invitation must be sent successfully.	Invitation details must be available: User Handler, Role, Status (Accepted, Pending, Canceled, Expired).
Cancel an invitation	The invitation is active.	• The invitation link to the invitation is revoked. • The invitation is no longer available to the user.
Resend an invitation	All details are in **Table 2.4.2**.	
Table 2.4.1		

Resend and compose an invitation Conditions		
Condition	**Action**	**Sender Allowed Operations**
The invited user didn't respond for 13 days.	The system automatically revokes the invitation link.	The sender is allowed to resend the same invitation or compose a new invitation to the same email. If the sender composed a new one, the old one is automatically canceled
If the sender canceled the invitation.	The system automatically revokes the invitation link.	The sender is allowed to compose a new invitation to the same email.
Otherwise, the invitation is still active, and they cannot compose a new one or resend an old one for the same email in the same project.		

Resend and compose an invitation Conditions
Table 2.4.2

2.4.5 The system must provide a status explaining to the sender the current state of the invitation. Detailed in **Table 2.4.3.**

Invitation States		
State	**Explain**	**Allowed Actions**
Pending Invite	The invitation is pending, but it still hasn't passed 13 days.	The sender can cancel it. The Sender can't resend it or compose a new one to the same email.
Expired invite	The invitation passed 13 days without accepting the invitation.	Sender can resend it or compose a new one to the same email. The invited user can accept the invitation
Accepted invite	The user accepted the invitation.	Sender can't resend it. Sender can't compose a new one. The invented user is now part of the project team.
Canceled	The inviter canceled the invitation	Sender can resend it or compose a new one to the same email. The invited user can accept the invitation
Table 2.4.3		

2.5 Invitation Acceptance and User Onboarding

Priority: Must have

Release: 1.0.0

User Requirement Specification

Invited users should be able to accept project invitations, whether they are existing users or new users without an account. The system should guide them through authentication or registration if required.

System Requirement Specification

2.5.1 The invited user will receive an email containing the following invitation details (based on the email template):

Subject: Project {project unique name} invites you to contribute to it as {role}

Message:

Message (if available)

User unique invitation link

Project owner name

2.5.2 The invitation link must display invitation details upon accessing the link, including:

- Project name
- Assigned role
- Current State (Valid, Expired, Canceled, Accepted)

2.5.3 The system must allow only the invited user to accept the invitation

2.5.4 The system must validate the invitation link to ensure that it is:

- Valid
- Not expired
- Not already used
- Canceled

2.5.5 The system must behave different based on the status of the invitation:

System behavior for invitation status	
Status	**System Behavior**
Already accepted	Show the invitation is already accepted and prevent link reuse.
Expired invitation	Show expiration message.
Invitation Canceled	Show the invitation was canceled.
Valid	Go with Invitation Acceptance Scenarios in requirement xxx
Table 2.5.1	

2.5.6 The system must allow the user to accept the invitation or to not take any action.

2.5.7 The system must handle the different invitation acceptance scenarios. Detailed in ***Table 2.5.2***

Invitation Acceptance Scenarios	
Scenario	**System Actions**
User is authenticated with the same invited email	The system must allow immediate acceptance of the invitation.
User is authenticated with another account	The system must 1. Asks the user to log out first 2. Redirects the user to the invitation page again.
User doesn't have an account with the same invited email	The system must: 1. Redirect the user to a registration interface 2. Allow account creation using the invited email 3. Redirect the user again to the invitation page.
User have an account but is not logged in	The system must: 1. Redirect the user to a login interface

Invitation Acceptance Scenarios	
	2. Redirect the user again to the invitation page.
Table 2.5.2	

2.5.8 The system must add the user to the project upon successful acceptance of the invitation.

2.5.9 The system must assign the predefined role to the user.

2.5.10 The system must update the invitation status to **Accepted**.

2.5.11 The system must prevent accepting the same invitation more than once.

2.6 Manage Roles and Team Members

Priority: Must have

Release: 1.0.0

User Requirement Specification

The Project Owner must be able to manage roles and team members in the project.

System Requirement Specification

2.6.1 The platform must allow the Project Owner to view the full list of project members and their roles.

2.6.2 The Project Owner must be able to remove a member from the project.

2.6.2.1 A confirmation prompt must appear before the removal is executed.

2.6.3 The project owner must be able to change the role of any member in the team.

2.7 Roles Permission

Priority: Must have

Release: 1.0.0

User Requirement Specification

Each team member in the project must have a role that describes what his mission and permissions.

System Requirement Specification

2.7.1 Each member in the project team must have at least and maximum one role, each role has allowed actions, and can't any actions not specified to this role. The roles with the allowed actions are detailed in ***Table 2.7.1***

Actions of each role	
Role	**Actions**
Project Owner	• Invite new Team member • Cancel invite • Resend invite • Change team members roles • Remove members from the team
Team Leader	• Bypass the update pipeline and merge code updates • Revert merged update and back to older version
Reviewer	• Move code for testing phase again with some notes • Move code for modifying phase again with some notes • Close the code and reject the update • Merge the update to production
Tester	• Create tests for the code • Mark test as done or failed for the code • Move code for modifying phase again with test results • Move the code for reviewing phase after passing all tests
Developer	• Create new update • Modify the update • Ask for testing • Link Bug to the update • Link Task to the update
Project Owner Team Leader Reviewer Tester Developer	• See all merged / reverted updates on production • See details of each merged / reverted updates on production • Create task • View task details • Change task status • Change task priority • Create bug • View bug details • Change bug status • Change task danger level
Table 2.7.1	

2.8 Developer Code Submission and Management

Priority: Must have

Release: 1.0.0

User Requirement Specification

Developers should be able to submit, edit, and manage code updates, track their status, and associate them with tasks and bugs while receiving feedback and notifications throughout the workflow.

System Requirement Specification

2.8.1 The system must allow developers to submit code updates by uploading new code files.

2.8.2 The system must support the following programming files. Detailed in ***Table 2.8.1***.

Programming Languages the system must support	
Programming language	**Files extension**
C Family	.c, .h, C#, .cpp, .cc, .cxx, .hpp, .h
java	.java
python	.py
Javascript Family	.js, .mjs, .cjs, .ts, .tsx
PHP	.php
Web files	.html, .css, .scss, .json, .xml
Table 2.8.1	

2.8.3 The system must display a success message upon successful code upload.

2.8.4 The system must allow developers to edit submitted code updates before entering the testing phase.

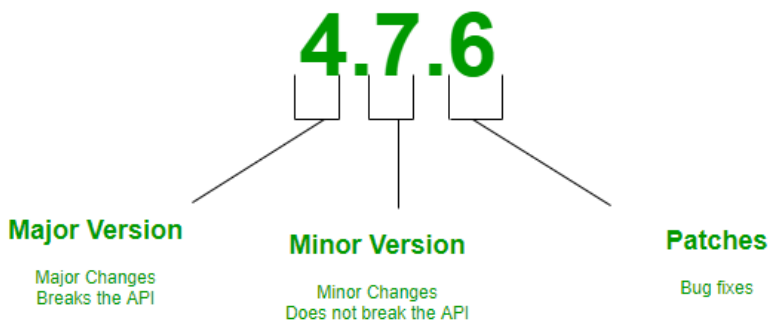
2.8.5 The system must allow developers to link code updates to:

- Tasks
- Bugs

2.8.6 The system must allow developers to add the following details about the new code update:

- Title
- Brief Description
- List of changes

- Version Type (Major, Minor, Patch)



2.9 Submission Status Tracking

Priority: Must have

Release: 1.0.0

User Requirement Specification

Developers should be able to monitor the status of their submissions to understand the current stage and required actions.

System Requirement Specification

2.9.1 The system must display the current status of each submission, based on the current phase.

2.9.2 The system must support the statuses for each phase transition. Detailed in ***Table 2.9.1***.

Transition between phases in pipeline	
Phase Transition	**Status**
Modify -> Testing	Under Testing
Review -> Testing	Under Retesting
Testing -> Modify	Test Failed, Waiting for Code Edit
Review -> Modify	Merge Rejected, Waiting for Code Edit
Testing -> Review	Under Review
Modify -> Merged	Successfully Merged, Pipeline Bypassed
Review -> Merge	Successfully Merged
Review -> Rejected	Closed

Table 2.9.1

2.9.3 The system must update submission status immediately after any decision or transition.

2.9.4 The system must notify the developer when the submission status changes. Detailed in ***Table 2.9.1***

2.10 Developer Feedback and Review Interaction

Priority: Should have

Release: 2.0.0

User Requirement Specification

Developers should receive feedback from testers and reviewers when code is moved again to the modification phase.

System Requirement Specification

2.10.1 The system must allow reviewers to provide feedback on code updates before moving the code again to the modification phase.

2.10.2 The system must allow testers to provide test results before moving the code again to the modification phase.

2.10.3 The system must notify developers when feedback is provided.

2.10.4 The system must allow developers to view all feedback and notes associated with their updates. Detailed in ***Table 2.10.1.***

Feedback on phase movement		
Source	**Phases Transition**	**Description**
Tester	Testing -> Modify	Test execution and failure feedback
Reviewer	Review -> Modify	Code quality and approval feedback
Table 2.10.1		

2.11 Test Monitoring for Developers

Priority: Must have

Release: 1.0.0

User Requirement Specification

Developers should be able to monitor test results to identify and fix issues in their code updates.

System Requirement Specification

2.11.1 The system must allow developers to view test results for their submitted updates.

2.11.2 The system must display Approve/reject status for each test case.

2.11.3 The system must notify developers when test results are available.

2.12 Code Editing and Resubmission Control

Priority: Must have

Release: 1.0.0

User Requirement Specification

Developers should be able to modify and resubmit their code updates when changes are requested from the reviewer or tests fail.

System Requirement Specification

2.12.1 The system must allow developers to edit code updates when:

- Tests Failed
- Reviewers request changes

2.12.2 The system must restrict code update changes once an update is moved out of the modifying phase

---

2.13 Workflow Automation and Phase Management

Priority: Must have

Release: 1.0.0

User Requirement Specification

The system must enforce an automated workflow for code updates, ensuring that updates move through predefined phases (Modify, Testing, Review, Merged, Rejected) in a controlled and consistent manner.

The system shall notify involved team members about changes in update status.

System Requirement Specification

2.13.1 The system must automatically move a submitted code update to the **testing phase** after submission.

2.13.2 The system must prevent a code update from moving to the **review phase** unless all associated tests are marked as passed.

2.13.3 The system must reset all test results associated with a code update whenever modifications are made to its code or the code is moved again to the **testing phase**.

2.13.4 The system must prevent a code update if the code:

- It is not in the **modify phase**
- The update is **rejected**

2.13.5 The system must prevent adding new tests for the code if the code is not in the **testing phase.**

2.13.6 The system should allow **team leaders** to override phase restrictions when necessary and merge an update directly by skipping the **testing** or **review phases**.

2.13.7 The system should allow **team leaders** to revert the update after it is merged.

2.14 Workflow Visibility for Developers

Priority: Should have

Release: 2.0.0

User Requirement Specification

Developers should have clear visibility of where their updates stand within the workflow pipeline.

System Requirement Specification

2.14.1 The system must highlight the current stage of each update.

2.14.2 The system must display previous and next stages for each update.

2.14.3 The system should provide visual indicators (e.g., progress bar, status labels).

2.15 Automatic Version Number Control

Priority: Could have

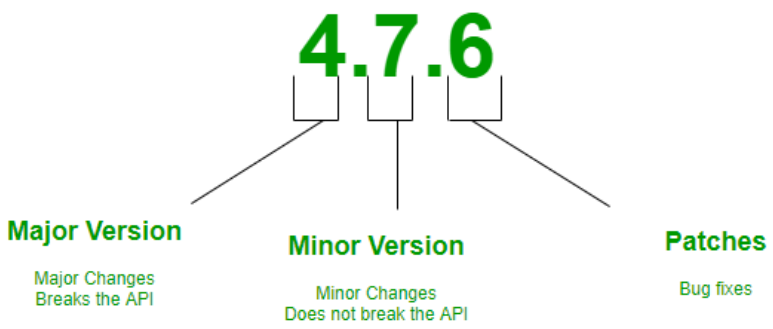
Release: 2.0.0

User Requirement Specification

The current version of the project must increase automatically when an update is merged, and when the update is reverted, the current version must be changed to match the latest previous unreverted update.

System Requirement Specification

2.15.1 After a successful update merger, the system must automatically increment the number of the current version based on the update type.



2.15.1.1 If the update type was patch, the system must increment the last digit of the previous version by one

2.15.1.2 If the update type was minor, the system must increment the second digit of the previous version by one

2.15.1.3 If the update type was major, the system must increment the first digit of the previous version by one

---

2.16 Release History and Version Tracking

Priority: Should have

Release: 2.0.0

User Requirement Specification

Team members should be able to track a complete history of all merged updates in the project, similar to a commit history, to understand changes over time and maintain traceability.

System Requirement Specification

2.16.1 The system must maintain a chronological history of all merged code updates within a project.

2.16.2 Each merged update or revert must be recorded as a **history entry**.

2.16.3 The system must store the following information for each history entry:

- Unique ID
- Update / Revert Title
- Author (user who submitted the update)
- Update/revert date and time
- Release Number
- Is the change an update or revert
- Reference for the updated or reverted release

2.16.4 The system must allow users to view the release/update history in a dedicated interface, that will show same stored information mentioned in ***2.16.3***

2.16.5 The system must display history entries in reverse chronological order (latest first).

2.16.6 The system should allow users to view detailed information for each history entry.

- Update / Revert Title
- Update / Revert Description
- Author (user who submitted the update)
- Update / revert date and time
- Effected tasks
- Effected bugs
- Release Number (in case of revert, will be = current release)
- Tests
- Is the change an update or revert
- If this change is merge:
    - Show the code
    - Show merge notes
- If this change is revert:
    - Show a reference for the reverted release
    - Show a reference for the current release
    - Show revert notes

2.16.7 The system should allow filtering the history or search by:

- Author
- Date
- Release version

2.16.8 The system must maintain history consistency even if updates are reverted.

2.16.9 If a merged update is reverted, the system must:

- Mark the corresponding history entry as **reverted**
- Record the revert action as a separate history entry

2.16.10 The system should allow these actions to the team members.
Detailed in ***Table 2.16.1***

Merged Updates History Actions		
Action	**Description**	**Allowed Role**
Revert	Mark update as reverted, create new entry to record this action in the history. And point to the latest non reverted update before the reverted one.	Team leader
View Details	Display full update information	All team members
Table 2.16.1		

2.17 Pipeline Monitoring

Priority: Must have

Release: 1.0.0

User Requirement Specification

Team leaders should be able to monitor the progress of updates across the pipeline to ensure smooth workflow execution.

System Requirement Specification

2.17.1 The system must provide an interface displaying pipeline stages:

- Modifying
- Testing
- Review
- Merged
- Closed

2.17.2 The system must display the status of each code update in real time.

2.17.3 The system must allow filtering updates by phase.

2.17.4 The system must highlight delayed or blocked updates.

2.17.4 The system must allow the team leader to show the submission details and code with details of the current phase.

2.18 Update Reversion Management

Priority: Must have

Release: 1.0.0

User Requirement Specification

Team leaders should be able to revert merged updates to undo problematic or unstable changes and restore system stability.

System Requirement Specification

2.18.1 The system must allow team leaders to revert any merged code update.

2.18.2 The system must restore the system state to the version prior to the reverted update.

2.18.3 The system must log the reversion action in the release history.

2.18.4 The system must automatically reopen all tasks and bugs linked to the reverted update.

2.19 Workflow Override Management

Priority: Should have

Release: 2.0.0

User Requirement Specification

Team leaders should be able to override workflow restrictions in urgent situations by skipping testing or review phases.

System Requirement Specification

2.19.1 The system must allow team leaders to skip the testing phase for a code update.

2.19.2 The system must allow team leaders to skip the review phase for a code update.

2.19.3 The system must require a justification note when skipping any phase.

2.20 Environment Management

Priority: Could have

Release: 2.0.0

User Requirement Specification

Team leaders should be able to manage environments and synchronize them to ensure consistency across deployments.

System Requirement Specification

2.20.1 The system must support multiple environments (Development, Testing, Production).

2.20.2 The system must allow only team leaders to synchronize other environments with the latest state of another environment when the team leader asks for that.

2.20.3 The system must allow only the team leader to create environments and choose what other environment to sync from.

2.20.4 The team leader must specify the following information about the environment while creating:

- Environment name
- Sync from (choose environment to sync from)

2.20.3.1 Upon successful environment creation, the new environment will have the same latest state from the “sync from” environment at the time of creation.

2.20.3.2 The system won’t sync the environment again until the team leader asks for that.

2.21 Environment Variables Management

Priority: Could have

Release: 2.0.0

User Requirement Specification

Team leaders should be able to create and manage environment variables for each environment to configure system behavior.

System Requirement Specification

2.21.1 The system must allow only the team leader to create environment variables for each environment with the following information:

- Variable Name
- Value

2.21.2 The system must allow the team leader to update and delete environment variables.

2.21.3 The system must restrict the view of environment variable management team members only.

2.21.4 The system must securely store environment variables.

2.22 Testing Workflow Management

Priority: Must have

Release: 1.0.0

User Requirement Specification

Testers should automatically receive testing requests when code updates are submitted, and be able to add and delete test cases and to change their state to passed or failed, before submitting the test results.

System Requirement Specification

2.22.1 The system must automatically notify testers when a new code update enters the **testing phase**.

2.22.2 The system must provide a dedicated interface for testers to view tests.

2.22.3 The system must allow testers to add new test case. And include the following information:

- Test name [required]
- Small description [required]
- Success Scenarios [required, at least 1]

2.22.4 The system must allow testers to remove existing test cases.

2.22.5 The system must allow testers to mark each test case result as:

- Pass
- Fail

2.22.6 Each new created test case will not have a results mark until the tester add pass or fail mark.

2.22.7 The system must allow testers to submit the test results after making sure all tests are marked and have results.

2.22.8 The test results cannot be submitted until having at least one test case and all test cases must have results.

2.22.9 After results are submitted the system must use test results to determine whether the update can proceed to the next phase.

2.22.10 After results are submitted the system must prevent updates from moving to review if any test case is marked as failed and move it to modify phase again.

2.22.11 After results are submitted the system must move updates to review if all test cases are marked as passed.

2.22.12 The system must allow testers to manage tests and the testing workflow only and only if the code is in the testing phase.

2.23 Re-Test and Consistency Control

Priority: Must have

Release: 1.0.0

User Requirement Specification

The system must clear the old results of tests when the code is submitted to test again.

System Requirement Specification

2.23.1 The system must automatically clear the old tests results when code is submitted to testing phase again and all tests will not have result mark.

2.23.2 The system must allow testers to start the testing workflow again. Detailed in *Requirement: 2.22*

2.24 Receive Updates from Testing

Priority: Must have

Release: 2.0.0

User Requirement Specification

The reviewer should be able to receive updates that passed testing so that they can review them.

System Requirement Specification

2.24.1 The system shall notify the reviewer when an update passes testing.

2.24.2 The system shall add the update to the review queue.

2.25 View Updates Waiting for Review

Priority: Must have

Release: 1.0.0

User Requirement Specification

The reviewer wants to see a list of all updates waiting for review so that they can perform review.

System Requirement Specification

2.25.1 The system shall provide a list of updates waiting for review.

2.25.2 The system shall allow the reviewer to view details and the code of any waiting updates .

2.25.3 The system shall allow the reviewer to view the code of the waiting updates to review it.

2.25.5 The system shall display the testing cases and results of the waiting updates.

2.26 Approve and Merge Updates

Priority: Must have

Release: 1.0.0

User Requirement Specification

The reviewer should be able to approve and merge updates with notes so that they become part of the code.

System Requirement Specification

2.26.1 The system shall allow the reviewer to approve updates.

2.26.2 The system shall allow the reviewer to add notes on the approved update.

2.26.3 The system will merge the approved updates to the dev environment and update the release number. Detailed in *Requirement: 2.15*.

4.26.4 The system shall update the update status to approved.

4.26.5 The system shall update the update status to merged after successful merging.

4.26.6 The system shall notify the developer that the update has been approved and merged.

2.27 Reject Updates

Priority: Must have

Release: 1.0.0

User Requirement Specification

The reviewer should be able to reject updates with comments and close the update request.

System Requirement Specification

2.27.1 The system shall allow the reviewer to reject updates.

2.27.2 The system shall allow the reviewer to add rejection comments.

2.27.3 The system shall close the update request.

2.27.4 The system shall update the update status to rejected

2.27.5 The system shall notify the developer of the rejection along with the rejection comments.

2.28 Send Updates Back to Testing

Priority: Must have

Release: 1.0.0

User Requirement Specification

The reviewer should be able to send updates back to testing with notes so that issues can be re-validated.

System Requirement Specification

2.28.1 The system shall allow the reviewer to send updates back to testing.

2.28.2 The system shall allow the reviewer to add notes.

2.28.3 The system shall update the update status to testing again.

2.28.4 The system shall notify testers and show the notes.

2.29 Send Updates Back to Modifying

Priority: Must have

Release: 1.0.0

User Requirement Specification

The reviewer should be able to send updates back to development.

System Requirement Specification

2.29.1 The system shall allow the reviewer to send updates back to development.

4.29.2 The system shall allow the reviewer to add notes for developers.

4.29.3 The system shall update the update status to modifying.

4.29.3 The system shall update the update status to modifying again.

4.29.4 The system shall notify the developer and show the notes.

2.30 Bugs Lifecycle Management

Priority: Could have

Release: 2.0.0

User Requirement Specification

Team members should be able to report and manage bugs linked to specific releases, collaborate by notifying other members, and ensure accurate tracking of bugs throughout the update lifecycle.

System Requirement Specification

2.30.1 The system must provide an interface for team members to report bugs associated with a specific release.

2.30.2 The system must require the following fields when reporting a bug:

- Title
- Description
- Danger Level (Normal, Urgent, Danger)

2.30.4 The system must support bug statuses:

- Open
- In Progress
- Completed

2.30.5 The system must automatically mark bugs as **completed** when a code update that references them is successfully merged.

2.30.6 The system must automatically reopen bugs if the associated code update is reverted.

2.30.7 The system must allow the team members to search and filter bugs by:

- Title
- Status (Open → In Progress → Closed)
- Danger Level

2.31 Notification Management

Priority: Must have

Release: 1.0.0

User Requirement Specification

Team members should receive notifications related to updates, bugs, tasks, and workflow transitions based on their role, to stay informed about project activities.

System Requirement Specification

2.32.1 The system must notify relevant team members upon each phase transition. Detailed in ***Table 2.31.1***.

Roles notified on phase transition	
Phase Transition	**Team members to be notified**
Modify -> Testing	All **Testers** in the team and the **Developer** who created the update.
Modify -> Merged	The **Developer** who created the update.
Testing -> Modify	The **Developer** who created the update.
Testing -> Review	All **Code Reviewers** in the team and the **Developer** who created the update.
Review -> Merge	The **Developer** who created the update, the **Team Leader** and the **Project Owner**.
Review -> Rejected Review -> Modify	The **Developer** who created the update.
Review -> Testing	All **Testers** in the team and the **Developer** who created the update.
Table 2.31.1	

2.31.2 The system must notify the **team leader** and **developers** when a new task or bug is **created**, **completed**, or **reopened**.

2.31.3 The system must notify the **team leader** and **developers** when a new task or bug is **created**, **completed**, or **reopened**.

2.31.4 The system must allow each team member to access a notification panel.

2.32 Task Lifecycle Management

Priority: Could have

Release: 2.0.0

User Requirement Specification

Team members should be able to create and track tasks related to project development.

System Requirement Specification

2.32.1 The system must allow any **team member** to create tasks within a project.

2.32.2 The system must allow only the task creator to delete or edit a task if it has not been completed.

2.32.3 The system must require the following information for task creation or editing:

- Title
- Description
- Priority (Normal, ASAP, Urgent)

2.32.4 The system must support task statuses:

- Open
- In Progress
- Completed

2.32.5 The system must automatically mark tasks as completed when linked updates are merged.

2.32.6 The system must automatically reopen tasks if linked updates are reverted.

2.32.7 The system must allow **developers** and the **team leader** only to change the task status.

2.32.8 The system must allow the task creator to mention only the **developers** and the **team leader** in the task.

2.32.9 The system must allow the team members to search and filter tasks by:

- Name
- Status
- Priority

2.33 Mentions and Collaboration

Priority: Won't Have

User Requirement Specification

Team members should be able to collaborate effectively by mentioning other users in different contexts.

System Requirement Specification

2.20.1 The system must allow users to mention other users using a unique identifier (e.g., @username).

2.20.2 The system must support mentions in:

- Bug reports
- Tasks

2.20.3 The system must notify mentioned users.

2.20.4 The system must highlight mentions within the interface.

1.34 Custom Permission System

Priority: Won't Have

User Requirement Specification

As a Project Owner, I want to define fully custom permissions per member so that I have fine-grained access control.

System Requirement Specification

1.34.1 This feature is out of scope for the current phase. Pre-defined roles (Developer, Tester, Reviewer, Team Leader) cover all needed scenarios.

1.34.2 May be reconsidered in a future sprint.

3.0 Non-functional Requirements

3.1 System Non-functional Requirements: General categories

---

1. Security

a. **Data protection:** Store sensitive data (passwords, tokens) using encryption techniques (e.g., hashing, SSL).
b. **Account protection:** Prevent unauthorized access using authentication and secure session management.
c. **Access & permission control:** Ensure role-based access control so users can only perform allowed actions.
d. **Secure communication:** All data exchanges must be protected using HTTPS protocols.

---

2. Performance

a. **Speed:** The system should load main pages within 2–3 seconds under normal conditions.
b. **Response time:** User actions (e.g., submitting updates, accepting invitations) should respond within 1–2 seconds.
c. **Concurrency:** The system should support multiple users working simultaneously without performance degradation.

---

3. Availability and Reliability

a. **Downtime:** The system should minimize downtime and recover quickly from failures.
b. **Uptime:** The system should maintain at least 99% availability annually.
c. **Reliability:** System operations (pipeline, updates, testing workflow) should execute consistently without failure.

---

4. Backup and Recovery

a. **Data backup:** The system must perform periodic automatic backups of all data.
b. **Recovery:** The system must restore data quickly in case of system failure or data loss.
c. **Version recovery:** Ability to restore previous versions of updates after reversion actions.

---

5. Scalability

a. **User scalability:** The system should handle increasing numbers of users and projects efficiently.
b. **Data scalability:** The system should support growth in stored data (files, updates, test cases) without performance issues.

---

6. Interoperability

a. **Integration:** The system should support integration with third-party tools (e.g., Git, CI/CD tools).
b. **API support:** Provide APIs for communication with external systems.

---

7. Usability

a. **Ease of use:** The system interface should be simple and intuitive for all roles (Developer, Tester, Reviewer, etc.).
b. **Learning curve:** New users should be able to use the system with minimal training.

---

8. Look and Feel

a. **User Interface:** The system should provide a clean and consistent UI design.
b. **Responsiveness:** The interface should be accessible across different devices and screen sizes.

---

9. Compatibility

a. **Browser compatibility:** The system should work on major browsers (Chrome, Firefox, Edge).

b. **Platform compatibility:** The system should run on different operating systems without issues.

---

10. Maintainability

a. **Code quality:** The system should follow clean code principles and modular design.
b. **Extensibility:** The system should allow easy addition of new features (e.g., new roles, pipeline stages).

If you want, I can next:

- Align these **exactly with your SRS sections (traceability)**
- Or convert them into a **professional Word document format**

4.0 Use-case Description

ID – Name	UC-01: Sign Up
Subsystem	Authentication & Account
Short Description	A user registers a new account in the system by providing the required personal information.
Preconditions	The user is not logged in.
Postconditions	A new user account is created and stored in the system (unverified state).
Actors / Initiators	Guest User
Trigger	The user navigates to the registration page.
Main Success Scenario	(1) The user clicks “Sign Up” (2) The system displays the registration form (3) The user enters required data (name, email, password) (4) The user submits the form (5) The system validates the input (6) If credentials are not used before and the data are valid, the system creates a new account in an unverified state (7) The system sends a verification email containing the verification link to the same email address. (8) The system notifies the actor to verify their account anytime. (9) The system redirects the user to the platform with restricted access until verification. (10) End use case.
Alternative Scenario	(6’) If input validation fails (wrong information or used email), the system shows errors → back to step 3

ID – Name	UC-02 – Verify an account
Subsystem	Authentication & Account
Short Description	The user verifies their account using a verification code sent via email.
Preconditions	User has a registered account in an unverified state.
Postconditions	The user account becomes verified.
Actors / Initiators	Registered User
Trigger	User opens request verification.

Main Success Scenario	(1) The user navigates the system (2) The user asks for account verification (3) The system sends a verification link to the user's email address. (4) The user receives the email and enters the link (5) The system marks the account as verified (6) End use case
Alternative Scenario	(3') The system failed to send a verification email to the user. (4') The system shows an error and asks the user to retry and move back to step 2 in the main success scenario. — —— —- — —— —- — —— —- — —— —- — —— —- — —— —- — —— — (4') The user didn't receive an email (5') The user requests a new email after 5 minutes (6') Back to step 3 in the main success scenario.

ID – Name	UC-03 – Login
Subsystem	Authentication & Account
Short Description	A registered user logs into the system.
Preconditions	User has a registered account.
Postconditions	User is authenticated and granted access.
Actors / Initiators	Guest User
Trigger	The user navigates to the login page.
Main Success Scenario	(1) The actor navigates the login page (2) The user enters their email and password (3) The system validates credentials (4) If the information is correct, the system authenticates the user (5) The user is redirected to the system
Alternative Scenario	(4') If credentials are invalid, the system shows an error and move the user back to step 2 in main success scenario.

ID – Name	US - Logout
Subsystem	Auth System
Short Description	A user can log out of the account
Preconditions	A user must be logged in with an account

Postconditions	The login session is cleared, and the user can't access the account unless they log in again. The user is redirected to the login page.
Actors / Initiators	User
Trigger	The user clicks logout
Main Success Scenario	(1) The user clicks logout (2) The system clears the session (3) The system redirects the user to the login page.

ID – Name	UC-05 – Create Project
Subsystem	Project Management
Short Description	A verified user creates a new project.
Preconditions	User account is verified.
Postconditions	The project is created and assigned to the creator user
Actors / Initiators	Verified User
Trigger	User opens the project creation page.
Main Success Scenario	(1) User clicks "Create Project" (2) System displays project form (3) User fills in the required data (4) System validates data (5) The system creates a project and assigns the creator as the owner of the project
Alternative Scenario	(4') If the information is not valid, show an error and go back to step 3

ID – Name	UC-6 – View Project Dashboard
Subsystem	Project Management
Short Description	User views project overview and details.
Preconditions	User is a project member.
Postconditions	Dashboard is displayed.
Actors / Initiators	Project owner
Trigger	User opens the project creation page.
Main Success Scenario	(1) User selects the project from a list of all projects they are part of

	(2) System loads project data (3) System displays dashboard

ID – Name	UC-7 – Invite User to Project
Subsystem	Invitation management
Short Description	A project leader can invite any activated user to the project
Preconditions	The user is a project leader or has permission. There is no active invitation to the same user from the same project.
Postconditions	The invitation is sent to the user. The invitation is recorded in the system for invitation management.
Error Situation	The invited user has an active invitation from the same project.
Actors / Initiators	Project owner
Trigger	The projectowner initiates the process to invite a developer to the project.
Main Success Scenario	(1) The inviter composes the invitation, including these information: ● User to invite email [required] ● A message [optional] ● A role [required] (2) The invited submits. (3) The system validates the inputs (4) If there is no active invitation to the user from the same project the system will do the following: ● The system generates unique invitation link and stores the invitation. ● The project's invitation list is updated, and the invitation state is marked as pending. ● The user will receive an invitation email with the invitation link.
Alternative Scenario	(4') If there is an active invitation to the user from the same project, the system will do the following: ● The system will notify the inviter that they can't send an invitation to this user now. ● The invitation is not sent. ● The project's invitation list will not be updated (5') End use case. — –— —- — –— —- — –— —- — –— —- — –— —- — –— — (3') If validation failed, the system shows an error and move the user back to step 2.

ID – Name	UC-8 – Manage the Sent project's invites
Subsystem	Invitation management
Short Description	A project owner can see all sent project invitations and their states, and manage them by canceling or resending.
Preconditions	The user must be the project owner
Postconditions	All changes and new state updates are stored and updated.
Actors / Initiators	Project owner
Trigger	The project owner initiates the process of management by navigating to the invitation management from the project dashboard.
Main Success Scenario	(1) The project owner navigates to the invitation management form project dashboard. (2) The system displays all sent invitations with state (canceled, pending, accepted) and details. (3) The actor can do the following action to each invitation: 3.a) Cancel a pending invitation 3.b) Resend inactive invitation (option is not available if it is active) (4) Upon successful action execution, all changes are stored, and status updates appear to the project owner.
Alternative Scenario	(3') The actor chooses to cancel an invitation (4') The system asks for confirmation (5') If the actor chooses to 5.a') confirm 5.b') cancel (6') 6.a') If the actor confirmed, the system marks this invitation as canceled and revokes the unique invitation link. 6.b') If the actor canceled, the system does nothing. (7') Back to step 4 in the main success scenario — —— —- — —— —- — —— —- — —— —- — —— —- — —— — (3') The actor chooses to resend an invitation (4') The system asks for confirmation (5') If the actor chooses to 5.a') confirm 5.b') cancel (6') 6.a') If the actor confirmed, the system generates new unique invitation link and reinvites the user again. 6.b') If the actor canceled, the system does nothing. (7') Back to step 4 in the main success scenario

ID – Name	UC-9 – Auto Expire Invitation
Subsystem	Invitation management
Short Description	The system automatically expires invitations after a period.
Preconditions	Invitation is pending.
Postconditions	Invitation becomes expired.
Actors / Initiators	System
Trigger	Expiration time reached.
Main Success Scenario	(1) The system checks the expiration time for each active invitation (2) If an active invitation has passed 13 days without accepting, the system revokes the invitation's unique URL. (3) The system marks the invitation as expired.

ID – Name	UC-9 – View Invitation
Subsystem	Invitation management
Short Description	An invited user accesses and views invitation details using the invitation link.
Preconditions	User has a valid invitation link.
Postconditions	Invitation details are displayed to the user.
Actors / Initiators	Invited User
Trigger	User clicks the invitation link.
Main Success Scenario	(1) User clicks the invitation link (2) System receives request with invitation token (3) System checks if the invitation is still valid (not expired or used) (4) The system checks if the current session belongs to the same invited user. (5) System retrieves invitation details (6) System displays project information, role, and message. (7') End use case
Alternative Scenario	(3') If the link is invalid / expired or has already been used, the system shows an error message. (4') End use case — –— —- — –— —- — –— —- — –— —- — –— —- — –— —

	(4') If the current session does not belong to the same invited user, the system asks the user to log out and then log in with the same account or create account with same email. (5') After a successful login / account creation, the system redirects the user again to the invitation page. (6') Back to step 2 from the main success scenario — –— —- — –— —- — –— —- — –— —- — –— —- — –— — (4') If there is no active session, the system asks the user to log in with the same account or create account with same invited email. (5') After a successful login / account creation, the system redirects the user again to the invitation page. (6') Back to step 2 from the main success scenario

ID – Name	UC-10 – Accept Invitation
Subsystem	Invitation management
Short Description	An invited user accepts an invitation and joins the project.
Preconditions	Invitation is valid and user is authenticated
Postconditions	User becomes a project member with assigned role.
Actors / Initiators	Invited User
Trigger	User clicks "Accept Invitation".
Main Success Scenario	(1) User chooses to accept invitation (2) System checks if the invitation is still valid (not expired or used) (3) System validates user identity with invitation (4) System marks invitation as accepted (5) System adds user to project (6) System assigns predefined role (7) System confirms successful onboarding (8) System revokes the invitation link to prevent reuse (9) End use case
Alternative Scenario	(2') If the link is invalid / expired or has already been used, the system shows an error message. (3') End use case — –— —- — –— —- — –— —- — –— —- — –— —- — –— — (3') If the current session does not belong to the same invited user the system asks the user to log out and then log in with the same account or create account with same email. (4') After a successful login / account creation, the system redirects

	the user again to the invitation page. (5') Back to step 2 from the main success scenario — —— —— — —— —— — —— —— — —— —— — —— —— — —— — (3') If there is no active session, the system asks the user to log in with the the same account or create account with same invited email. (4') After a successful login / account creation, the system redirects the user again to the invitation page. (5') Back to step 2 from the main success scenario

ID – Name	UC-11 – Manage Team Members
Subsystem	Project management
Short Description	Project owner manages project team members, by viewing members, change roles and removing them.
Preconditions	Use must be project owner
Postconditions	Team member list is updated if changes are made.
Actors / Initiators	Project owner
Trigger	User opens the team management interface.
Main Success Scenario	(1) User navigates to team management section (2) System retrieves and displays team members (3) For each team member actor can do one of these actions: 3.a) Remove a team member 3.b) Change role of a team member (4) System asks the actor to 4.a) confirm action 4.b) cancel action (5) Ubon successful execution, the system apply changes and update the list.
Alternative Scenario	(4') If the actor choose cancel, the system will don nothing. (5') End use case

ID – Name	UC-12 – Restrict Unauthorized Actions
Subsystem	Role Permissions
Short Description	The system blocks unauthorized operations.
Preconditions	User lacks required permissions.

Postconditions	Action is prevented.
Actors / Initiators	System
Trigger	Unauthorized action attempt.
Main Success Scenario	(1) System detects insufficient permissions (2) System blocks action (3) System shows error or warning

ID – Name	UC-14 – Submit Code Update
Subsystem	Workflow Pipeline Management
Short Description	A developer creates and submits a complete code update, including uploading files, adding details, linking related items, and selecting the version type.
Preconditions	User with developer role in the project
Postconditions	● Code update is created and stored ● Update is linked to related tasks/bugs (if provided) ● Version type is assigned ● Update enters the workflow modifying phase
Actors / Initiators	Developer
Trigger	Developer selects “Create / Submit Update”
Main Success Scenario	(1) Developer opens the “Submit Update” interface (2) System displays the update creation form (3) Developer enters update details (title, description) (4) System checks if details are provided (5) Developer uploads code files (6) Upon successful upload, the system validates and stores the uploaded files (7) The system checks if at least one file is uploaded (8) Developer selects version type (patch/minor/major) (9) System checks if version type is selected (10) Developer optionally links the update to related tasks (11) Developer optionally links the update to related bugs (12) Developer submits the update (13) System validates all input data (14) System creates the code update record (15) System associates uploaded files with the update (16) System links selected tasks and bugs (17) System places the update into the workflow in the modifying

	phase (18) System confirms successful submission
Alternative Scenario	(4') If details are not specified, the system prompts the user to enter them. (5') Back to step 4 in the main success scenario — -— —- — -— —- — -— —- — -— —- — -— —- — -— —- — -— — — (6') If the files upload fails, the system shows errors (7') Back to step 5 in the main success scenario. — -— —- — -— —- — -— —- — -— —- — -— —- — -— —- — -— — — (7') If there is not at least one fiel uploaded, the system prompts the user to upload one. (8') Back to step 5 in the main success scenario. — -— —- — -— —- — -— —- — -— —- — -— —- — -— —- — -— — — (9') If version type is not selected, the system prompts the user to select one (10') Back to step 8 in the main success scenario.

ID – Name	UC-15 – Edit Code Update
Subsystem	Workflow Pipeline Management
Short Description	The developer edits an existing code update.
Preconditions	● User with developer role in the project ● The code update is in the modifying phase.
Postconditions	● Changes are saved and applied to the code update
Actors / Initiators	Developer
Trigger	The developer selects "Edit code update"
Main Success Scenario	(1) Developer opens the "Edit code Update" interface (2) The developer can change details (except version type), upload, and delete files (3) The user clicks submit (4) The system validates the new details (5) The system validates if at least one file is uploaded (6) Upon successful validation, the system stores the new updates (7) Upon successful store, the system notifies the developer.
Alternative Scenario	(4') If details are not valid, the system prompts the user to enter them. (5') Back to step 2 in the main success scenario

	— (2’) If files upload fails, the system shows errors (3’) Back to step 2 in the main success scenario. — (7’) If the strong fails, the system asks the user to retry. (8’) Back to step 3 in the main success scenario

ID – Name	UC-16 – Submit code update for testing
Subsystem	Workflow Pipeline Management
Short Description	The developer can submit a code update to the testing phase when it is ready.
Preconditions	● User with developer role in the project ● The code update is in the modifying phase.
Postconditions	● Update is moved to the testing phase ● Testers get notified ● The developer can’t modify the update.
Actors / Initiators	Developer
Trigger	The developer submits the update for testing.
Main Success Scenario	(1) The developer triggers the action to submit the update for testing. (2) The system moves the update from the modifying phase to the testing phase. (3) The system notifies all testers of a new update requesting a test. (4) The system automatically resets any previous test results (5) The system prevents any developer from modifying the update.

ID – Name	UC-17 – Track Update Status
Subsystem	Workflow Pipeline Management
Short Description	The developer and team leader can track the current status of an update.
Preconditions	● Update exists in any phase.
Postconditions	● Status is displayed.
Actors / Initiators	Developer, Team Leader
Trigger	The developer or Team Leader views the update.
Main Success Scenario	(1) System retrieves status

	(2) System displays the current status

ID – Name	UC-18 – Testers Receive Testing Request
Subsystem	Workflow Pipeline Management
Short Description	Tester receives a request to test an update.
Preconditions	Update enters testing phase.
Postconditions	Tester is notified.
Actors / Initiators	Developer
Trigger	Update moves from modifying to testing.
Main Success Scenario	(1) System notifies testers of the test request (2) Testers get notification of the test request

ID – Name	UC-19 – View Updates ready for test
Subsystem	Workflow Pipeline Management
Short Description	The tester views the code update to be able to perform tests
Preconditions	The update is in the testing phase.
Postconditions	The tester viewed the details and code of the update.
Actors / Initiators	Tester
Trigger	Viewing code update
Main Success Scenario	(1) The tester opens the update (2) The system fetches the full details and displays them (3) The System fetches the uploaded files and displays the code

ID – Name	UC-20 – Add test case
Subsystem	Workflow Pipeline Management
Short Description	The tester is able to add a test case with details and success scenarios for the update that is waiting for testing.
Preconditions	Test case for the update must exist
Postconditions	Test case is added
Actors / Initiators	Tester
Trigger	Adding test case

Main Success Scenario	(1) The tester triggers adding a test case. (2) The system opens a form asking for the required information in requirement 2.22 (3) The user submits the test (4) The system validates the information (5) Upon successful validation, the system stores the test with an initial state of no result. (6) Upon successful test store, the system shows success message and updates tests list.
Alternative Scenario	(5’) If validation failed, the system prompts the user to enter valid information (6’) Back to step 3 in main success scenario — (6’) If storing failed, the system shows error message and asks the tester to retry again. (6’) Back to step 3 in main success scenario

ID – Name	UC-20 – Delete test case
Subsystem	Workflow Pipeline Management
Short Description	The tester is able to delete a test case.
Preconditions	The update is in the testing phase.
Postconditions	Test case is deleted
Actors / Initiators	Tester
Trigger	Delete test case
Main Success Scenario	(1) The tester triggers delete a test case. (2) The system asks for confirmation (3) Once the tester confirmed the test is deleted with all its results and relates.
Alternative Scenario	(3’) If the tester canceled, the system does nothing (4’) End use case.

ID – Name	UC-21 – Add test result
Subsystem	Workflow Pipeline Management
Short Description	The tester is able to add/change the test result to a test case.
Preconditions	Test case for the update must exist

Postconditions	Test case result is added/changed
Actors / Initiators	Tester
Trigger	Add test case result
Main Success Scenario	(1) The tester chooses a test case to add a result (2) The tester chooses the result as passed or failed. 2.1) If the test result fails, the tester must specify feedback. (3) The system directly updates the test case result to the selected result.

ID – Name	UC-22 – Submit test result
Subsystem	Workflow Pipeline Management
Short Description	The tester is able to submit all test results so the system decides what the next phase.
Preconditions	● At least 1 test case exists ● All test cases have results
Postconditions	The results are submitted, and the system moves the update to the decided phase.
Actors / Initiators	Tester
Trigger	Submit test results
Main Success Scenario	(1) The tester triggers the test results submission. (2) The system asks for confirmation (3) The tester confirms (4) The system decides what is the next phase 4.a) If at least 1 test case result is failed, the update is moved to the modifying phase with the test results. 4.b) If all test cases are passed, the update is moved to the reviewing phase with test results. (5) The system shows what phase the update moved to. (6) Testers are not able to perform any actions for the tests. (7) The system notifies the people who are responsible on the next phase 6.a) If the update moved to the review phase, the reviewers are notified 6.b) If the update is moved to modify, the developer is notified.
Alternative Scenario	(3’) If the tester canceled, the system does nothing. (4’) End use case.

ID – Name	UC-23 – View Updates ready for review
Subsystem	Workflow Pipeline Management
Short Description	The tester views the code update to be able to perform tests
Preconditions	The update is in the testing phase.
Postconditions	The tester viewed the details and code of the update.
Actors / Initiators	Tester
Trigger	Viewing code update
Main Success Scenario	(1) The tester opens the update (2) The system fetches the full details and displays them (3) The System fetches the uploaded files and displays the code

5.0 Architecture

5.1 Auth Service

Responsibility
Everything related to authentication and account lifecycle, nothing else.

Owns
- User credentials (email, password hash)
- Account verification state
- Sessions/tokens (JWT, refresh tokens)

Operations
- Sign up
- Login/logout
- Email verification
- Resend verification
- Token refresh

Used By
- API Gateway
- All services (via token validation)

5.2 Project Service

Responsibility
Project as a business entity

Owns
- Project data (name, description, logo, version)
- Project ownership

Operations
- Create project
- Get project
- List user projects

5.3 Invitation Service

Responsibility
Full invitation lifecycle

Owns
- Invitation entity
- Invitation token/link
- Status (Pending, Accepted, Expired, Canceled)

Operations
- Send invitation
- Cancel invitation
- Resend invitation
- Validate invitation
- Accept invitation

Important Logic
- Expiration (13 days)
- Duplicate prevention
- Link generation

Talks To
- Project Service (project existence)
- Notification Service (email)
- Team & Role Service (Check if user is not part of this project, and check if the sender is the owner)

5.4 Team & Role Service

Responsibility
Authorization inside a project (Who can do what and where)

Owns
- Project membership
- Roles per member

Operations
- Add member (after invitation accepted)
- Remove member
- Change role
- Get members & roles

- Check permission (CRITICAL)

Used By
- API Gateway
- All services (via token validation)

5.5 Pipeline Service

Responsibility
Everything about code submissions and the modifying phase

Owns
- Update state (Modify, Testing, Review, etc.)
- Transition rules
- Phase validation

Operations
- Move to testing
- Move to review
- Move to modifying
- Reject/merge/revert
- Validate transitions
- Current State of update

Critical Rules
- Cannot move to review unless tests are passed
- Reset tests on modification
- Enforce phase restrictions

Used By
- Developers, Testers, Reviewers, Leader

Talks to
- Modifying Phase Service, Testing Phase Service, Review Phase Service
- Notification Service

5.6 Modifying Phase Service

Responsibility
Everything about code submissions and modifying phase

Owns

- Code update entity
- Uploaded files (or references to storage)
- Metadata (title, description, version type)
- Links to tasks/bugs

Operations

- Create update
- Edit update (only in modify phase)
- Upload/delete files
- Link tasks/bugs
- Show update details

Must NOT handle

- Workflow transitions
- Testing
- Review decisions

Used By

- Developers

Talks to

- Team & Role Service (to check if user has developer role)
- Workflow / Pipeline Service

5.7 Testing Phase Service

Responsibility
Testing lifecycle

Owns

- Test cases
- Test results

Operations

- Add/delete test case
- Mark pass/fail
- Submit results

Must NOT handle

- Workflow transitions
- Code Submission
- Reviewing decisions

Used By

- All tests must be marked
- At least one test required
- Determines next phase

Used By

- Testers

Talks to

- Team & Role Service (to check if user has tester role)
- Workflow / Pipeline Service

5.8 Reviewing Phase Service

Responsibility
Code review decisions

Owns

- Review decisions
- Review notes

Operations

- Approve update
- Reject update
- Send back to testing
- Send back to modify

Must NOT handle

- Workflow transitions
- Code Submission
- Testing decisions

Used By

- Reviewers

Talks to

- Team & Role Service (to check if user has reviewer role)
- Workflow / Pipeline Service

5.9 Release & Version Service

Responsibility
Versioning + history

Owns
- Version number
- Release history
- Revert tracking

Operations
- Increment version
- Store history entry
- Handle revert

Logic
- Patch / minor / major increment
- Revert restores previous state

5.10 Bug Service

Responsibility
Bug lifecycle

Owns
- Bugs
- Status (Open, In Progress, Completed)
- Danger level

Operations
- Create bug
- Update status
- Link to update
- List Bugs

Logic
- Auto close on merge
- Reopen on revert

5.11 Task Service

Responsibility
Task lifecycle

Owns
- Tasks
- Status
- Priority

Operations
- Create task
- Update task
- Change status
- List tasks

Logic
- Same as bugs (merge/revert effects)

5.12 Notification Service

Responsibility
Centralized communication

Owns
- Notifications
- Delivery (email, in-app)

Operations
- Send notification
- Store notifications
- Retrieve user notifications